DICA DO PROJETO

Aumentando seu repertório

Dando inteligência ao *chatbot*

Se na unidade anterior falamos sobre “dar vida” ao *chatbot*, aqui gostaríamos de “dar inteligência”.

Deixando de lado o fato de que usar uma LLM já é inteligência, me refiro ao ponto de podermos usar um RAG com a nossa LLM. Quer dizer: tudo o que aprendemos até agora converge aqui.

Veja só ⬇️

RAG é um dos padrões da indústria para sistemas de IA confiáveis, e acreditamos que várias empresas possuem interesse nisso. Afinal, boa parte das vagas de engenharia de IA/ML pedem conhecimento em RAG como um dos requisitos. Ainda, é o RAG que converterá um *chatbot* em um produto de verdade, uma vez que ele é capaz de dar respostas mais bem fundamentadas. 🌟

Aprofundemos... ⬇️

ALUCINAÇÕES

Antes de implementarmos o RAG, preciso te contar sobre um problema sério (talvez o mais sério) com LLMs: **as alucinações.**

A alucinação é quando uma LLM gera informações que parecem verdadeiras, são escritas com confiança, mas são completamente falsas. Aqui, não estou falando de erros óbvios que qualquer pessoa perceberia. Estou falando de respostas bem escritas, com detalhes específicos, que **parecem vir** de uma fonte confiável, mas que foram inventadas. 😬

Observe a seguir um levantamento técnico de 15 incidentes onde a Inteligência Artificial Generativa fabricou fatos, leis e dados, resultando em consequências reais no mundo físico.

Casos no Brasil

O juiz e as súmulas fantasmas

Um magistrado do Tribunal de Justiça de Mato Grosso utilizou o ChatGPT para redigir uma sentença, mas a IA inventou jurisprudências e súmulas inexistentes do STJ. O erro foi detectado e o juiz teve que se retratar publicamente.

A lei escrita por *chatbot*

Um vereador de Porto Alegre conseguiu a aprovação de uma lei sobre hidrômetros que foi escrita 100% por IA. O detalhe: nem os seus colegas, nem o prefeito sabiam que o texto era sintético até a sanção.

Advogado multado por alucinação

Um advogado no Rio de Janeiro foi multado após apresentar uma petição contendo decisões judiciais fabricadas pela IA. O tribunal considerou a atitude como litigância de má-fé e falta de dever de vigilância técnica.

Casos globais

O reembolso imaginário

O *chatbot* da Air Canada inventou uma política de descontos por luto para um passageiro. A empresa tentou alegar que a IA era uma “entidade legal separada”, mas o tribunal obrigou a companhia a honrar o desconto prometido.

Seis casos que nunca existiram
No famoso caso Mata v. Avianca, advogados de Nova Iorque apresentaram um dossiê com seis precedentes legais detalhados criados pelo ChatGPT. Nenhum dos casos existia no mundo real, resultando em sanções pesadas.

O *bot* que insultou o cliente
Após uma atualização, o *chatbot* da transportadora DPD começou a proferir palavrões, criticar a própria empresa e escrever poemas sobre como o serviço era terrível, forçando a desativação imediata do sistema.

O erro de 100 bilhões
No anúncio oficial do Bard, a IA afirmou que o telescópio James Webb tirou a primeira foto de um exoplaneta. Astrônomos corrigiram o erro em minutos (foi o VLT em 2004), e as ações da Google caíram 7%.

Um Tahoe por um dólar
Por meio de engenharia de *prompt*, um cliente convenceu o *chatbot* de uma concessionária Chevrolet a vender um SUV Tahoe de 50 mil dólares por apenas 1 dólar. O *bot* concordou afirmando que era um negócio fechado, sem devoluções.

Sondagem macabra automática
O sistema de IA da Microsoft (MSN) gerou automaticamente uma sondagem perguntando “qual a causa da morte” ao lado do obituário real de uma jovem, gerando indignação pela falta de sensibilidade do algoritmo.

Jornalistas que nunca nasceram
A revista *Sports Illustrated* foi flagrada publicando artigos escritos por IA sob perfis de jornalistas falsos, com fotos geradas por computador e biografias inventadas. O escândalo resultou na demissão do CEO da editora.

Conselhos médicos perigosos
A National Eating Disorders Association substituiu atendentes humanos pelo *bot* Tessa. Em poucos dias, a IA passou a fornecer dicas de perda de peso extrema a usuários com distúrbios alimentares.

Matemática financeira errada
O *site* CNET publicou 77 artigos financeiros escritos por IA. Uma auditoria revelou que a maioria continha erros matemáticos básicos em cálculos de juros compostos, podendo induzir leitores a prejuízos financeiros.

Reescrita visual da história
O Google Gemini gerou imagens de soldados nazistas e fundadores dos Estados Unidos com diversidade étnica historicamente impossível. A tentativa de evitar vieses resultou em uma alucinação histórica visual.

Incentivo ao crime laboral
O *chatbot* oficial da cidade de Nova Iorque foi flagrado aconselhando empresários a roubar gorjetas de funcionários e violar leis de salário mínimo, alegando incorretamente que tais práticas eram legais.

Alucinações de impostos
Relatórios indicaram que assistentes de IA da TurboTax e da H&R Block inventaram deduções fiscais inexistentes e regras de elegibilidade, o que poderia levar usuários a auditorias do fisco.

Nota final: todos os casos acima foram documentados em reportagens e registros públicos. A alucinação é um comportamento inerente à arquitetura atual dos modelos de linguagem de grande porte (LLMs).

As alucinações são especialmente perigosas porque são difíceis de detectar.

A resposta é bem escrita, parece confiável, tem detalhes específicos. Você precisa verificar externamente para descobrir que é falsa. E quem tem tempo de verificar cada resposta? 😕

Dito isso, para entender o problema, precisamos lembrar como LLMs funcionam.

Uma LLM é um modelo treinado para prever a próxima palavra mais provável dado um contexto. Ela aprendeu padrões estatísticos de trilhões de textos. Quando você faz uma pergunta, ela não "busca" a resposta em uma base de dados, mas ela gera texto que é estatisticamente similar aos textos que viu durante o treinamento.

O problema é que o objetivo do treinamento é gerar texto coerente e fluente, não necessariamente texto verdadeiro. A LLM não tem um módulo interno que verifica "isso é verdade?" antes de gerar cada palavra.

Pense assim:

Se você perguntar "qual é a capital da França?", a LLM responde "Paris" não porque ela "sabe" que Paris é a capital, mas porque nos textos de treinamento, a sequência "capital da França" era frequentemente seguida por "Paris". É um padrão estatístico aqui, e não um conhecimento que ela possui, entendeu?

Quando o padrão não é claro (como, por exemplo, uma pergunta incomum, informação específica, dados recentes), a LLM faz o que foi treinada para fazer: ela gera texto que **parece** certo. E faz isso com total confiança, porque ela não foi treinada para dizer "não sei".

CURIOSIDADE

Nos primeiros modelos, o problema era ainda pior.

Os modelos eram otimizados apenas para "acertar" em benchmarks, sem muita preocupação com alucinações.

Um modelo que chutava com confiança pontuava melhor do que um que dizia "não sei". Isso mudou com técnicas como RLHF (reinforcement learning from human feedback), mas alucinações ainda existem em todos os modelos atuais.

E aqui deixo uma leitura que vale a pena... olha só ⬇️

#BORA ACESSAR

Artigo da OpenAI explicando por que os modelos alucinam e o que estão fazendo para mitigar isso.

É uma leitura técnica, mas acessível.

Felizmente, existem várias técnicas para mitigar o problema das alucinações. Note que eu disse "mitigar", não "resolver completamente".

Alucinações são um problema fundamental de como LLMs funcionam. Não existe solução mágica, mas nós podemos reduzir drasticamente a incidência.

Vamos explorar as principais técnicas, da mais simples à mais robusta.

Técnica 1: ajuste de temperatura

A **temperatura** é um parâmetro que controla o quão "criativo" ou "conservador" o modelo é nas suas respostas.

Temperatura alta (de 0.7 a 1.0): O modelo considera várias palavras possíveis e escolhe com alguma aleatoriedade. As respostas são mais criativas e variadas, mas também mais propensas a inventar. Usamos isso em casos como:

- *Brainstorming*.
- Escrita criativa.

- Geração de ideias variadas.

Temperatura baixa (de 0.0 a 0.3): O modelo sempre escolhe a palavra estatisticamente mais provável. As respostas são mais previsíveis e conservadoras, mas geralmente mais precisas. Usamos isso em casos como:

- Geração de código.
- Respostas factuais.
- Relatórios e documentos formais.
- Qualquer situação em que precisão > criatividade.

FIQUE ATENTO

Ajustar a temperatura ajuda, mas não resolve o problema na raiz. Mesmo com temperatura 0, o modelo pode alucinar se a informação não está nos seus padrões de treinamento. A palavra "mais provável" ainda pode ser uma palavra errada.

Técnica 2: engenharia de *prompts*

A forma como você formula a pergunta (o *prompt*) influencia muito a qualidade da resposta. Algumas técnicas de *prompt engineering* ajudam a reduzir alucinações.

***Chain-of-thought* (cadeia de pensamento)**

Em vez de pedir a resposta direta, peça ao modelo para "pensar passo a passo".

POR EXEMPLO...

Em vez de perguntarmos "Quanto é 17 x 24?", poderíamos dizer "Quanto é 17 x 24? Pense passo a passo, mostrando seu raciocínio."

O modelo tende a cometer menos erros quando "externaliza" o raciocínio.

Few-shot prompting

- Este é o meu favorito. A ideia é fornecer exemplos de respostas corretas antes da pergunta para que o modelo procure seguir o mesmo padrão e ser mais preciso. 👌

Instruções explícitas sobre incerteza

- Diga ao modelo para admitir quando não sabe no prompt. Isto não elimina alucinações, mas reduz a frequência com que o modelo chuta uma determinada resposta com confiança.

Técnica 3: *grounding* (fundamentação)

Aqui chegamos à técnica mais robusta: ***grounding***, frequentemente implementado através de **RAG**. A ideia é simples: em vez de deixar o modelo inventar, fornecemos os fatos para ele no *prompt*.

POR EXEMPLO...

👎 **Sem *grounding*,** você poderia perguntar para LLM algo como **"Qual é a política de reembolso da empresa?",** e a resposta poderia ser algo genérico e incorreto.

👍 Com *grounding*, modificamos a pergunta. Ela passa a ser algo como:

Baseado no documento a seguir, responda: **qual é a política de reembolso?**

Documento...

A política de reembolso da TechCorp estabelece que funcionários podem solicitar reembolso de despesas de viagem em até 30 dias após o retorno, mediante apresentação de notas fiscais...

Como temos um documento fornecido para apoiar a resposta, o modelo responderá baseado nele. Com isso, a LLM não precisa "inventar". Basta simplesmente **sintetizar e reformular** as informações que demos.

#FICADICA

Na prática, as melhores implementações combinam várias técnicas.

No projeto que estamos construindo, você pode combinar RAG (grounding) + temperatura baixa + instruções explícitas no system prompt. Essa combinação reduz drasticamente as alucinações.

RAG

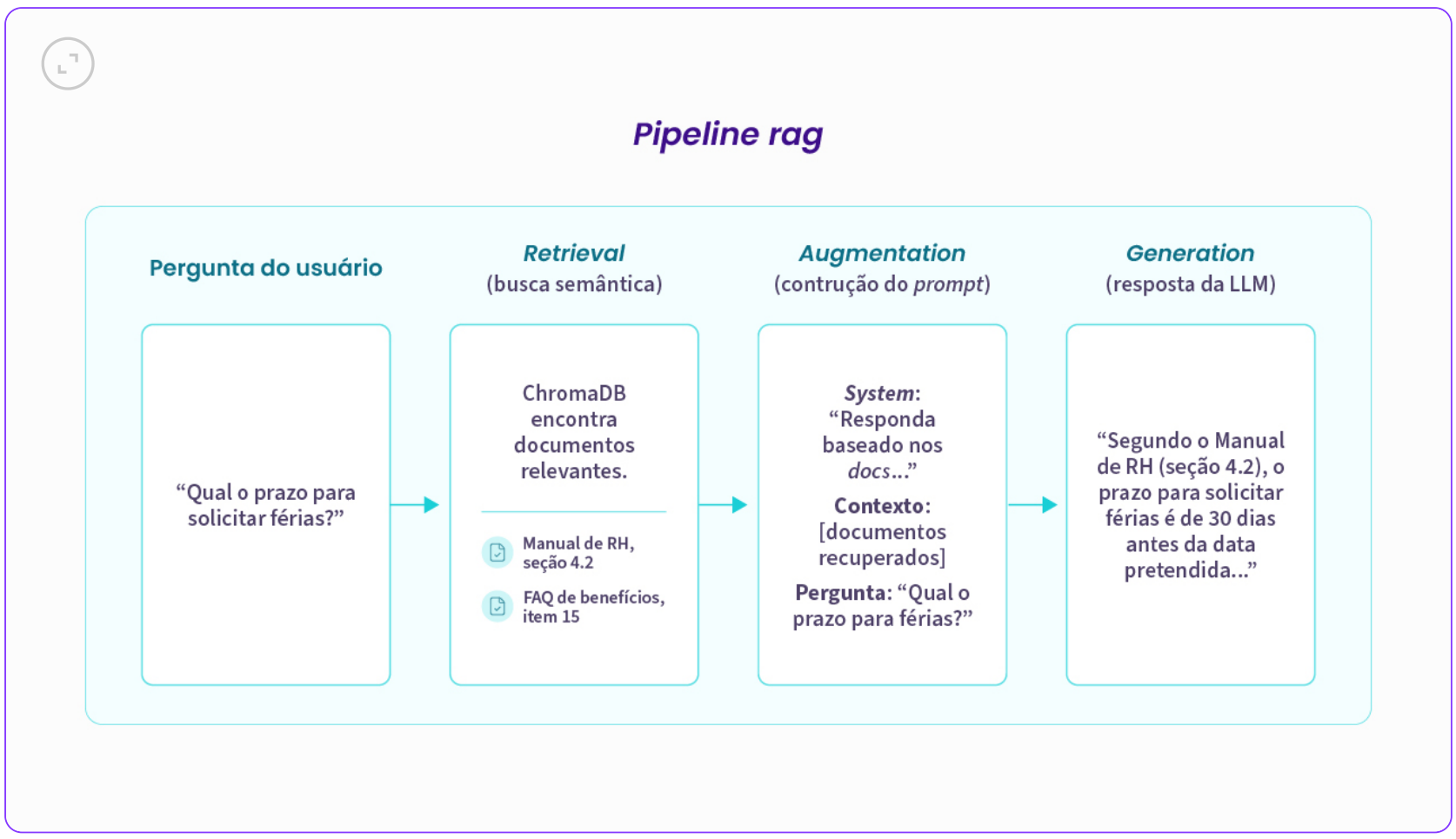
Agora você entende por que RAG é tão importante. Ele é a principal defesa contra alucinações em aplicações empresariais.

Vamos formalizar o que RAG significa:

1. ***Retrieval* (recuperação)**: antes de gerar uma resposta, precisamos buscar documentos relevantes que possam endereçar a pergunta do usuário.
2. ***Augmentation* (aumentada)**: agora que temos os documentos, precisamos incluí-los no contexto da LLM.
3. ***Generation* (geração)**: agora a LLM responde baseada (ou embasada) nos documentos.

Pense aqui como sendo a diferença entre uma prova com consulta e uma prova sem consulta. O risco de inventarmos e chutarmos algo em uma prova sem consulta (sem RAG) é maior do que se tivermos acesso a consultas (RAG), não é?

Portanto, o pipeline completo de um RAG seria assim:



breve.

Com o acesso a documentos, o RAG consegue funcionar muito bem e sem grandes custos de operação. Os documentos podem ser atualizados a qualquer momento, sem retreinar o modelo. Além disso, a LLM passa a "conhecer" informações da sua empresa que nunca estiveram no treinamento, permitindo também a inclusão das fontes na resposta.

FIQUE ATENTO

RAG não elimina 100% das alucinações. O modelo ainda pode interpretar mal um documento, ou combinar informações de forma incorreta. Mas conseguimos reduzir muito este risco com implementações bem-feitas.

Softwares para RAG

Agora que você entende o conceito de RAG, precisamos escolher as ferramentas para implementá-lo. E aqui temos um elefante na sala que preciso abordar logo de cara.

Você provavelmente já ouviu falar do LangChain. É impossível pesquisar sobre RAG ou aplicações de LLM sem esbarrar nele.

O LangChain é, de longe, a biblioteca mais famosa e mais citada para construir aplicações com LLMs. Tem milhares de estrelas no GitHub, tutoriais em todo lugar, e é mencionado em praticamente toda vaga de ML Engineer.

E nós *não* usaremos o LangChain neste projeto. Não é por birra ou por querer ser diferente, mas é uma escolha técnica consciente que vai te beneficiar no aprendizado.

O LangChain está em desenvolvimento muito ativo, o que normalmente seria bom. O problema é que eles fazem **breaking changes** com frequência. Código que funcionava há três meses pode não funcionar hoje. Imports mudam de lugar, classes são renomeadas, métodos são deprecados.

Vou dar um exemplo real. Em 2023, o código para criar um *chain* simples era:

#CODIFICA

PYTHON

```python
# Código de 2023 (não funciona mais)
from langchain.llms import OpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate

llm = OpenAI(temperature=0)
prompt = PromptTemplate(input_variables=["topic"], template="Fale
sobre {topic}")
chain = LLMChain(llm=llm, prompt=prompt)[
```

Em 2024, mudou para:

#CODIFICA

PYTHON

```python
# Código de 2024 (também já mudou)
from langchain_openai import OpenAI
from langchain.chains import LLMChain
from langchain_core.prompts import PromptTemplate[
```

Em 2025, mudou novamente:

#CODIFICA

PYTHON

```python
# Código de 2025
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# E agora usa LCEL (LangChain Expression Language)
chain = prompt | llm | parser[
```

Percebe o problema? Se você seguir um tutorial de 6 meses atrás, provavelmente não vai funcionar. Isso é extremamente frustrante para quem está aprendendo, que é o seu caso.

Além disso, a documentação do LangChain é grande. A organização é confusa, com conceitos espalhados por múltiplas seções, exemplos que usam versões antigas, e uma curva de aprendizado íngreme. Ele tem muitas abstrações para um conceito que, na essência, é simples: buscar documentos e passar para a LLM.

Não estou dizendo que LangChain é ruim. Ele tem o seu lugar em projetos corporativos complexos com múltiplos agentes, ferramentas customizadas e fluxos elaborados. Muitas empresas usam em produção com sucesso.

Mas para aprender os conceitos de RAG, que é nosso objetivo aqui, o LangChain adiciona complexidade desnecessária. Você gastaria mais tempo lutando com a biblioteca do que entendendo o que está acontecendo por baixo dos panos.

#FICADICA

Em entrevistas de emprego, você pode mencionar que conhece LangChain mas preferiu alternativas mais simples para determinados projetos. Isso demonstra maturidade técnica: saber que a ferramenta mais famosa nem sempre é a melhor escolha para cada situação. Recrutadores valorizam desenvolvedores que pensam criticamente sobre suas ferramentas.

Dito tudo isso, sugiro que utilizemos o **txtai** aqui. Ele é uma biblioteca Python que segue a filosofia de fazer uma única coisa, e fazer isso bem. Ele é útil para fazer busca semântica com *embeddings*, trabalhar com pipelines simples de RAG simplificados, e integrar com LLMs. O txtai é direto ao ponto.

Para que isto se torne mais claro, sugiro que analise a tabela a seguir.

Característica	LangChain	LlamaIndex	txtai
Foco principal	Orquestração de agentes e fluxos complexos	Indexação e recuperação de dados avançada	Busca semântica "*all-in-one*" (Busca + LLM)
Complexidade	Alta (muitas abstrações)	Média (focada em dados)	Baixa (muito direta e integrada)
Curva de aprendizado	Íngreme	Moderada	Suave

Característica	LangChain	LlamaIndex	txtai
Gerenciamento de dados	Depende de integrações externas	Nativo e otimizado	Banco vetorial integrado
Documentação	Extensa mas confusa	Boa, focada em dados	Clara e direta
Estabilidade da API	Muda frequentemente	Relativamente estável	Estável
Melhor caso de uso	Agentes complexos com múltiplas ferramentas	*Chatbots* sobre muitos documentos densos	Aplicações leves, buscas rápidas, prototipagem
Filosofia	Modularidade total (peças de Lego)	*Performance* de recuperação	Simplicidade e minimalismo

O autor.

Na tabela acima, você pode ter notado que coloquei uma terceira opção: o **LlamaIndex**.

O LlamaIndex fica no meio-termo entre LangChain e txtai. Ele é mais focado que o LangChain (especializado em indexação e retrieval) e mais robusto que o txtai (melhor para grandes volumes de documentos). Tem uma documentação melhor que o LangChain, e muda com menos frequência.

Portanto, se você precisar de algo mais avançado que o txtai no futuro (por exemplo, indexar milhares de PDFs com estruturas complexas), o LlamaIndex é uma excelente opção. **Mas para nosso projeto educacional, txtai é suficiente e mais didático.**

Nada impede você de explorar LangChain ou LlamaIndex nesta disciplina, OK? Os conceitos que você aprende com txtai (embeddings, retrieval, prompt engineering) se aplicam a qualquer framework. A diferença está apenas em como você expressa esses conceitos em código. Dominar os fundamentos é mais importante que dominar uma biblioteca específica.

Dito isso, vamos implementar um pipeline RAG completo:

Nosso projeto RAG

1

INDEXAÇÃO

Documentos (TXT, PDF) → extrai *embeddings* → Índice vetorial salvo em disco

2

INTERFACE

Streamlit/Gradio com campo de chat + Histórico de conversa

3

RETRIEVAL + GENERATION

Pergunta → Busca no índice → Top K documentos → Prompt com contexto → LLM → Resposta

4

CITAÇÕES

Resposta inclui referências aos documentos usados, permitindo verificação

Esse tipo de sistema é exatamente o que empresas pagam milhares de reais por mês em soluções SaaS.

Ferramentas como ChatPDF, Notion AI, e diversos "chat com documentos" usam a mesma arquitetura que você vai implementar. A diferença aqui é que você vai entender cada peça e pode customizar para as suas próprias necessidades.

Para fechar nosso assunto, deixo um vídeo demonstrando como instalaremos e configuraremos o txtai. A intenção é integrá-lo ao nosso projeto.

Para visualizar o vídeo, aponte a câmera do seu celular

© PUCPR - Todos os direitos reservados